

MOBILE ROBOTICS

Simulation Problem Set

PART 01 — ROBOT MODELLING

Design your robot's URDF

Build a URDF for a differential-drive robot of your choice. Decide on your own dimensions, shapes, and link structure. Get it spawning correctly in RViz before moving on.

- Where does your base_link sit on the robot, and why did you choose that location?
- How did you decide on the geometry for each link - box, cylinder, sphere?
- What does the joint type for each wheel need to be, and why?

Resources:- [URDF — ROS 2 Documentation: Foxy documentation](#)

[Create a URDF with ROS2 \[1H Crash Course\]](#)

[Robotics # Build-a-Bot using URDF | by Dilip Kumar | Feb, 2026](#)

Add collision and inertia

Extend your URDF with collision geometry and inertial properties. Try launching in Gazebo without them first, observe what breaks.

- Does your collision geometry need to match your visual geometry exactly? When might you simplify it?
- What happens in simulation if you omit the inertial tag entirely?
- How did you calculate or estimate the mass and moments of inertia for your shapes?

Resources:- [How to Calculate Inertia values of your Robot and spawn in Gazebo](#)

PART 02 — SIMULATION IN GAZEBO

Drive your simulated robot

Add the differential drive Gazebo plugin and get your robot moving with teleop. Tune the plugin parameters to match your robot's actual dimensions.

- What parameters did you need to get right for the odometry to be accurate?
- What happened when you drove the robot without fixing the caster wheel friction?
- How does the plugin decide which way is "forward"?

Resources:- [Setting up a robot simulation \(Gazebo\) — ROS 2 Documentation](#)

Explore the coordinate frame tree

Drive your robot around in Gazebo and observe the TF frames in RViz. Try setting the fixed frame to odom, then drive in a loop and return to start.

- What do the map, odom, and base_link frames each represent?
- After driving a loop, is base_link back at the odom origin? Why or why not?
- Which transform is smooth but drifts, and which is accurate but may jump?

PART 03 — SENSORS

Add a simulated LiDAR

Add a 2D LiDAR to your robot's URDF and configure the Gazebo ray sensor plugin. Visualise the scan data in RViz as you drive around.

- Where on the robot did you mount the LiDAR, and what influenced that decision?
- What parameters did you need to configure, range, angle, update rate?
- What is the frame_id in the LaserScan message, and why does it need to match your URDF?
- Build a small obstacle course in Gazebo. Can you navigate by watching the scan in RViz?

Add a simulated camera

Add an RGB camera to your robot. Get the image feed appearing in RViz or rqt_image_view. Pay attention to how camera coordinate frames work.

- What does RPY rotation mean? What are the different formats of images used?
- Install the image_transport plugins. What compressed formats become available, and how would you republish between them?

Resources:- [How to Add Sensor Plugins to Gazebo](#)

All the above resources were provided to give you an idea of the tags and features available in Gazebo and the capabilities of the URDF format.

The ideal way to create a bot for use in Gazebo is to design a CAD model in SolidWorks and then export it to URDF using its plugin.

[Convert SolidWorks to URDF for ROS2 - Full Tutorial](#)

PART 04 — SLAM & NAVIGATION

Map your simulated environment with SLAM

Run slam_toolbox in online asynchronous mode. Drive around your Gazebo world and build a map. Then save it and reload it for localisation only.

- What does the map → odom transform tell you, and who is publishing it?
- After returning to your start position, how far has odom drifted from the map origin?
- What is the difference between saving and serialising your map? When would you use each?
- Can you reload your saved map and localise without re-mapping?

Resources:-

<https://www.youtube.com/watch?v=ZaiA3hWaRzE&list=PLunhgkrRNRhYAffV8JDIFoatQXuU-NnxT&index=17>

Navigate autonomously with Nav2

Launch Nav2 on top of your SLAM setup. Set a 2D goal pose and let the robot navigate to it. Add an obstacle mid-route and observe how the planner responds.

- What is a cost map, and how does it differ from your SLAM occupancy grid?
- What happens when you set the inflation radius too large for your environment?
- How does twist_mux let you take manual control mid-navigation? What determines priority?
- Try setting multiple waypoints. What changes in how Nav2 behaves?

Resources:- [Making robot navigation easy with Nav2 and ROS!](#)

Hardware Part

Setting up the Raspberry Pi 5

Get your Raspberry Pi 5 running Ubuntu and ROS2, and connect it to your development machine on the same network.

- What Ubuntu version did you choose and why? Is it compatible with your ROS2 version?
- How are you connecting to the Pi — SSH, monitor, or something else? What are the tradeoffs?
- What does use_sim_time need to be set to on the real robot, and why does it matter?
- How do you ensure ROS2 nodes on the Pi and your dev machine can discover each other?

Design your power system

Plan the full power circuit for your robot before buying or wiring anything. Your robot needs to run the Pi5, motors, and at least one sensor from a single battery.

- What battery chemistry and configuration (voltage, capacity, C-rating) did you choose? Justify it against your estimated current draw.
- What voltage does the Pi 5 require, and how are you supplying it cleanly from your main battery?

- Where does a fuse go in the circuit, and what rating did you choose? What does it protect?
- How did you estimate the total current draw of your system? Show rough calculations.
- What is the difference between a linear regulator and a switching regulator, and which did you use for your 5V rail?

MOTORS & CONTROL

11. Choose and justify your motors

Select the drive motors for your robot. Your choice will affect everything downstream — the motor driver, the battery, the chassis size, and how accurately you can control speed.

- What type of motor did you choose- brushed DC, brushless, stepper? Why is that type suitable for a differential drive robot?
- What voltage and stall current does your motor have? How does this affect your selection of battery and motor driver?
- Do your motors have encoders? If yes, how will you use them? If not, what are the consequences for speed control?
- What gear ratio are you using, and how does it affect the trade-off between speed and torque?

12. Wire up your motor driver and controller

Get your motors spinning under control from the Pi. You can use an Arduino or another microcontroller as an intermediate motor controller.

- What motor driver did you choose, and is its current rating sufficient for your motors' stall current?
- How is the Pi communicating with the motor controller - USB serial, I2C, GPIO PWM? What influenced that decision?
- Can you command each motor independently to go forward, backward, and stop? How did you verify this?
- If you are using encoders, can you read back speed or position? How are you converting raw encoder counts into useful units?

SENSORS ON THE REAL ROBOT

13. Integrate a real LiDAR

Connect the provided physical 2D LiDAR to your robot and have it publish scan data that RViz can display.

- How did you reliably assign the correct serial port to the LiDAR across reboots?
- Drive the robot slowly and watch the scan in RViz. Does the scan update smoothly? What causes lag or drops?

14. **Integrate a real camera**

Connect the official Pi camera to the Pi and get a live image stream visible from your development machine.

- What resolution and frame rate are you running at, and how does that affect network bandwidth?
- Is the image compressed or raw? How would you switch between them using `image_transport`?
- How did you set the `frame_id` so that the image data aligns with your robot's TF tree?

Integration

15. **Run SLAM on the real robot**

Drive your assembled robot around a real room and build a map using `slam_toolbox`. Compare what you see in RViz with the physical environment.

- How does the real odometry drift compare to what you saw in simulation? What physical factors cause the difference?
- Were there parts of the room the robot struggled to map? What sensor or algorithmic limitations explain this?
- Save your map. Can you reload it and localise the robot in the same room the next day?

16. **Autonomous navigation on the real robot**

Get Nav2 running on the physical robot using your saved map. Send it a goal and let it navigate without your help.

- What had to change in your launch files or parameters compared to the simulation setup?
- How did the robot handle dynamic obstacles, a person walking across its path, a chair moved since the map was made?
- Did you need to tune any Nav2 parameters for the real robot? Which ones, and what changed?
- Reflect on the full stack: what is the weakest link between the sensor, the map, the planner, and the actuators?

Part C — Wireless Control & Teleoperation

Understand the network load

Connect your development device and the Raspi6 using a camera. Wireless bandwidth is limited. Some ROS2 topics are tiny; others, like raw camera images, can saturate a network link entirely.

- Estimate the data rate of your camera topic at your chosen resolution and frame rate. Is raw or compressed more practical over Wi-Fi?
- What happens to your LiDAR scan rate if the network drops packets? How would you notice this in RViz?
- Which topics are critical for safe teleoperation, and which are just for monitoring? What would you drop first if bandwidth became a problem?

Handle stamped vs. unstamped twists

ROS2 control's diff drive controller can accept either Twist or TwistStamped on its command topic. The choice has practical consequences.

- What extra information does a TwistStamped carry compared to a plain Twist? When is that information actually useful?
- If your teleop tool only publishes plain Twist but your controller expects TwistStamped, how do you bridge the gap?
- What could go wrong if the clock on the Pi and the clock on your dev machine are not synchronised when using stamped twists?
- Try changing `use_stamped_vel` in your controller YAML. What breaks, and what do you need to fix to make it work again?

Visualise the robot using Foxglove Studio

- Create a ROS 2 bag file and visualise the robot on Foxglove Studio
- Figure out all the features that Foxglove has to offer for robot visualisation

Send commands from a phone or tablet

Browser-based tools make it possible to control the robot from a mobile device. Explore what that looks like in practice and where the limits are.

- Can you send velocity commands to the robot from a touchscreen? What tool or approach did you use?
- What is the difference between using on-screen joystick buttons versus a physical Bluetooth gamepad paired to a tablet?
- What happens to the robot if your browser tab loses focus or the network drops mid-command? How would you make this safer?
- If you were deploying this robot in a real environment, what would you want in a remote operator interface that none of the current tools provide?